Design of Hardware Accelerators

Lab Lesson 2

Elia Lazzeri
Dipartimento di Elettronica, Informazione e Bioingegneria
Politecnico di Milano

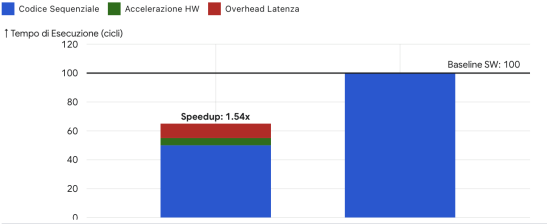
April 12, 2026

Today's Laboratory Session

- Context from Lab 2: We successfully brought up the base NEORV32 SoC and evaluated software execution performance.
- Even with caches and hardware multipliers enabled, pure software execution of complex algorithms is limited by the sequential nature of the instruction pipeline.
- Today's Goal: Transition from a homogeneous system to a Heterogeneous System-on-Chip (HSoC).
- Hands-on Task: Identify a compute-intensive software kernel, partition the design, and implement a dedicated hardware accelerator to execute that kernel.

Why Implement Accelerators?

- Hardware can exploit massive spatial parallelism, executing multiple operations per clock cycle, unlike a CPU's temporal execution.
- Energy Efficiency (Dark Silicon)
- Overall system speedup is bottlenecked by the sequential portion of the code. We accelerate the "hot loops" (e.g., 90% of execution time spent in 10% of the code).
- We trade silicon area (LUTs/FFs) and design complexity for execution speed and power efficiency.



NEORV32 Custom Functions Subsystem (CFS)

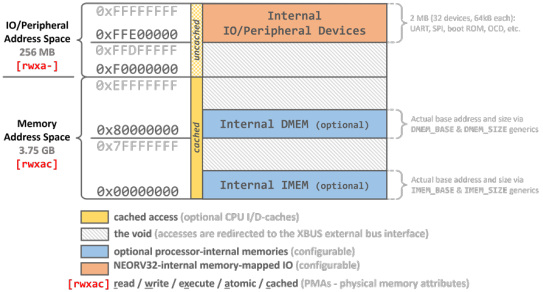
- What is the CFS? A tightly coupled, memory-mapped interface designed specifically for adding custom hardware to the NEORV32 without modifying the core CPU pipeline.
- Architecture:
  - Directly connected to the internal processor bus (Wishbone).
  - Provides a pre-configured VHDL template (neorv32_cfs.vhd).
- Acts as a sandbox. You design the internal datapath and Finite State Machine (FSM), and the CFS handles the bus interface complexities.

Memory-Mapped I/O

- The hardware accelerator does not have special instructions. It looks like standard memory to the CPU.
- Address Space: The CFS registers are mapped to a specific base address in the processor's memory map.
- Software View (C Code):
  - Interaction is done via standard C pointers writing to specific memory addresses.
  - Example: NEORV32_CFS REG[0] = data_in; (Sends data to HW)
  - Example: result = NEORV32_CFS REG[31]; (Reads data from HW)
- Hardware View (VHDL):
  - The VHDL code detects a write-enable signal on the Wishbone bus for a specific register index and captures the data into internal flip-flops.

Memory-Mapped I/O

- Hardware can exploit massive spatial parallelism, executing multiple operations per clock cycle, unlike a CPU's temporal execution.



CPU-Accelerator Synchronization

Question

Hardware takes time to compute. How does the CPU know when to read the result safely?

- Method 1: Polling (Busy-Waiting)
  - The CPU continuously reads a specific CFS "status" register (e.g., while(NEORV32_CFS REG[0] == BUSY);).
  - Pros: Simple to implement. Cons: Wastes CPU cycles and power. Overhead can kill the speedup if the HW task is too small.
- Method 2: Interrupts
  - The accelerator asserts a hardware IRQ line when finished. The CPU can do other work and jump to an Interrupt Service Routine (ISR) when the data is ready.
- Method 3: Fixed-Cycle Delay
  - If the hardware pipeline has a strictly deterministic latency (e.g., exactly 5 cycles), the CPU can simply insert NOPs or execute 5 cycles of independent instructions before reading.

Development Workflow

- Interface Definition: Map out how the CFS registers will be used (e.g., Reg 0-17 for Data IN, Reg 18 for Control/Start, Reg 19-27 for Data OUT, Reg 28 for Status).
- Hardware Design (VHDL): Implement the algorithmic datapath and the control FSM within the neorv32_cfs.vhd template.
- Software Wrapper (C): Write a C function that abstracts the HW interaction (writes inputs, triggers execution, polls status, reads outputs).
- Co-Simulation: Use GHDL to run the C program on the simulated CPU and verify the bus transactions and accelerator waveforms.
- Synthesis: Compile the full HSoC for the Tang Nano 9k and upload the bitstream.

Hands-on Task: Matrix Multiplication Accelerator

- Hardware Objective: Implement a fixed-size Matrix Multiplication unit (10x10 matrices) inside the CFS.
  - Implement the Multiply-Accumulate (MAC) logic and the controlling FSM in VHDL.
  - Plan your memory map carefully!
- Software Objective: Write a C program to validate the accelerator.
  1. Define two test matrices.
  2. Send the matrices to the CFS, trigger the computation, and retrieve the result.
  3. Verify the hardware result against a pure software C implementation of the same multiplication.

Final Comparison: SW vs. HW Implementation

Note

You must extract two key metrics for both the pure C version and your new HW-accelerated version.

1. Performance (Execution Time):
   - Use the mtime system timer to count the exact clock cycles taken by the C function vs. the HW-accelerator function (including the data transfer overhead).
   - Calculate the real Speedup.
2. Area Occupation:
   - Check the Gowin EDA synthesis report. How many additional LUTs and FFs did your accelerator consume compared to the baseline core from Lab 1?

Question

Did the performance gain justify the area cost? If the matrices were 100x100 instead of 3x3, how would your design need to change?